Splat Editor: a Viewer and Editor for 3D Gaussian Splatting Scenes

Lê Anh Kiệt [a]

[a] *International University, Vietnam National University Ho Chi Minh City, Vietnam* Student ID: ITCSIU24047

P R O J E C T I N F O

Keywords:
3D Gaussian Splatting
Camera and projection
Lighting and shadows
Physics simulation
ImGui interface

A B S T R A C T

This project builds Splat Editor, a program to view and edit 3D Gaussian Splatting (3DGS) scenes in Python and OpenGL 3.3. The program implements the 3DGS rendering pipeline itself: it reads the PLY file, projects ellipsoids to ellipses, sorts by depth and blends with alpha. On top of this, the project uses the six technique groups of the course: transformations, camera, texture, lighting, animation and interaction. The user can select, delete, undo and save the file.

1. Introduction and gaussian theory

1.1. Introduction

3D Gaussian Splatting (3DGS) shows a 3D scene with millions of soft ellipsoids, called gaussians [2]. The scene is trained from photos taken around an object (Fig. 1). The resulting `.ply` file always has floating gaussians. So we need a program to view and clean the file. SuperSplat [3] does this in the browser. I wrote a similar editor in Python and OpenGL 3.3.

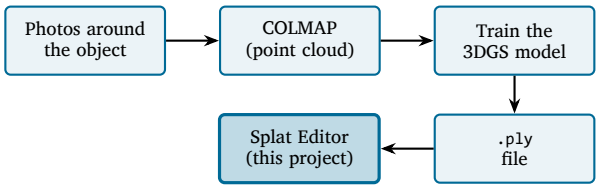


Fig. 1. The workflow from photos to Splat Editor. COLMAP [1] finds the camera positions.

The project has two parts. Part one is the 3DGS rendering pipeline, from the file to the shader. Part two is the six technique groups of the course (Table 1).

Table 1. Technique groups and sections.

Group	Main content	Section
T1	layers, matrix **M**	2
T2	view/projection matrices, EWA, blending, cameras	3
T3	character skin texture	4
T4	SH colour, Phong, shadow map	5
T5	day–night, collision, player	6
T6	selection brush, delete/undo, ImGui, HUD	7

1.2. Program structure

The source is split by role: `core` holds only data and math, so it does not depend on OpenGL; `render` holds all OpenGL calls and shaders; `ui` builds the interface; and `app.py` ties the parts together. Figure 2 lists representative files only.

```
SplatOpenGL/
|-- main.py              # entry point: python main.py [scene.ply]
|-- requirements.txt     # numpy, PyOpenGL, glfw, imgui-bundle
|-- report/              # LaTeX report
`-- splat_editor/
    |-- app.py           # main loop, ties everything together
    |-- core/            # data and math, no OpenGL
    |   |-- splats.py    # gaussian arrays
    |   |-- layer.py     # one PLY file + transform + selection
    |   |-- scene.py     # the set of layers
    |   |-- math3d.py    # matrices V, P, Euler angles, quaternions
    |   |-- lighting.py  # sun, lamps, day--night
    |   |-- physics.py   # voxel collision, jumping
    |   |-- avatar_pose.py  # character joint angles
    |   |-- lod.py       # level-of-detail selection
    |   `-- history.py   # undo
    |-- io/              # file reading / writing
    |   |-- ply.py       # read / write PLY
    |   `-- codec.py     # PLY columns <-> Splats
    |-- editor/          # editing
    |   |-- editor.py    # open, save, delete, select
    |   `-- brush.py     # brush selection
    |-- render/          # OpenGL
    |   |-- splat_renderer.py  # draws gaussians (2 passes)
    |   |-- sorter.py    # depth sort (background thread)
    |   |-- camera.py    # orbit / fly camera
    |   |-- avatar_renderer.py # draws the character
    |   |-- shader.py    # compiles GLSL
    |   |-- shaders/     # splat_prep.vert, splat.vert/.frag, ...
    |   `-- textures/    # skinzom.png
    `-- ui/              # ImGui interface
        |-- window.py    # GLFW window
        |-- input.py     # keyboard, mouse
        `-- sections.py  # control panels
```

Fig. 2. Directory tree of the program (main files only).

1.3. Input data

The input is a `.ply` file made by 3DGS training. Training starts from the COLMAP point cloud and puts one gaussian at each point. Then it repeats three steps. It renders the gaussians. It compares the image with the real photo. It updates the parameters of every gaussian. It also removes useless gaussians and splits gaussians that are too large.

The file has two parts: a text header and a binary body. The header lists the properties. The body has one record per gaussian. Every value is a 32-bit float. Table 2 lists the properties. With SH of degree 3, one record has 62 values, or 248 bytes. Fig. 3 shows the first rows of a file, exported as CSV.

Table 2. Properties of one gaussian in the PLY file.

Property	Count	Meaning
`x, y, z`	3	centre μ
`nx, ny, nz`	3	normal, always 0, not used
`f_dc_0..2`	3	base colour (SH degree 0)
`f_rest_*`	45	SH coefficients of degrees 1–3
`opacity`	1	raw opacity
`scale_0..2`	3	logarithm of the size
`rot_0..3`	4	rotation quaternion

```
x,y,z,nx,ny,nz,f_dc_0,                                f_dc_1,f_dc_2,
2.98395634,0.64003098,1.57549512,0,0,0,0.0184020139,-0.12875317,-0.452952206,
4.22914267,-0.663039207,-0.523675323,0,0,0,-0.150325835,-0.135893673,-0.325479239,
1.13421345,0.410973638,3.93819642,0,0,0,0.346005678,0.0445601828,-0.405238956,
0.604746521,2.51691461,-0.371717632,0,0,0,-0.157206044,-0.176818103,-0.256703675,
0.642261147,-1.39632988,-4.3651309,0,0,0,-0.227364123,-0.334438473,-0.433887929,
-4.2221036,-0.695849061,0.247175366,0,0,0,0.412634581,0.405996054,0.111131459,
```

```
opacity,scale_0,scale_1,scale_2,rot_0,rot_1,rot_2,rot_3
-0.0695948228,-0.0550651364,-0.0382503457,-0.0237467252,0.0131118316,0.0629977509,-0.0449319854,-0.
0.000325970963,-0.0160180721,0.00357675459,0.0473092273,0.0389139801,-0.0540050268,-0.0265164431,-0
0.0612790994,0.0957935154,-0.0631214827,0.0422011353,0.0244775601,0.0491610095,0.0470059477,0.07206
-6.13165855,-2.71730924,-2.80323267,-2.86005092,0.876973867,0.112431385,0.0801429227,-0.0182835758
-5.53735924,-2.7295742,-2.76182842,-2.74456096,0.885511041,0.101086691,-0.0186818913,-0.0949812829
-0.0379554592,0.00524016004,0.0189723969,-0.0380355604,-0.0966132209,-0.00303976983,-0.0181281436,-
```

Fig. 3. The first six gaussians of a PLY file, shown as CSV. Top: position, normal and base colour. Bottom: opacity, scale and rotation. The `f_rest` columns in between are hidden.

The program maps a binary file straight into memory. So a file with millions of gaussians opens almost at once. Fig. 4 shows a small capture of a person. The blurry blobs around the body are the floating gaussians that the editor must remove. The main test scene is a street with about 5 million gaussians (Fig. 5).

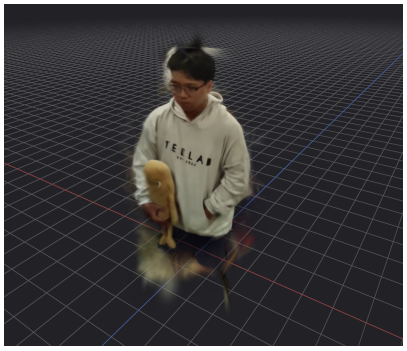


Fig. 4. A PLY file of a captured person. Floating gaussians are visible around the body and near the ground.

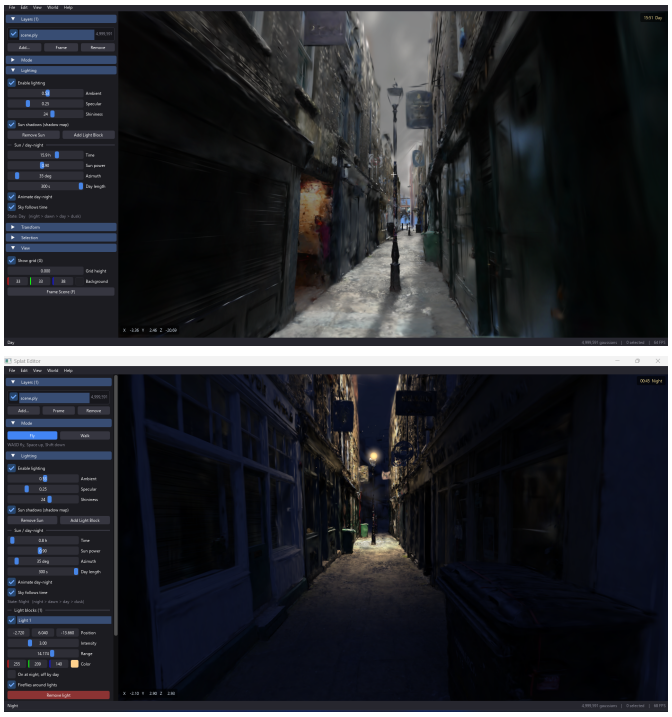


Fig. 5. A scene of about 5 million gaussians in Splat Editor, by day (top) and by night (bottom).

1.4. Gaussians and the PLY file

Each gaussian has a position $\boldsymbol{\mu}$, a colour $\mathbf{c}$, an opacity α, a size $\mathbf{s}$ and a rotation $\mathbf{q}$ (quaternion). Its density falls off smoothly from the centre. We can picture it as an ellipsoid (Fig. 6). The PLY file stores raw values. The program decodes them on load:

- Size: $s_i = \exp(\texttt{scale_i})$.
- Opacity: $\alpha = 1/(1 + e^{-\texttt{opacity}})$.
- Colour: $\text{RGB} = 0.5 + C_0\,\texttt{f_dc}$, with $C_0 \approx 0.2821$.
- Rotation: the quaternion `rot_0..3` is normalised to length 1.

On save, the program does the reverse.

Listing 1. `io/codec.py`, `SplatCodec.decode`: decoding the raw PLY values

```
SH_C0 = 0.28209479177387814
colors = 0.5 + SH_C0 * np.stack([col('f_dc_0'), col('f_dc_1'), col('
↪ f_dc_2')], axis=1)
alphas = 1.0 / (1.0 + np.exp(-col('opacity')))
scales = np.exp(np.stack([col('scale_0'), col('scale_1'), col('
↪ scale_2')], axis=1))
q = np.stack([col('rot_0'), col('rot_1'), col('rot_2'), col('rot_3')],
↪  axis=1)
length = np.linalg.norm(q, axis=1, keepdims=True)
q = q / np.maximum(length, 1e-12)
```

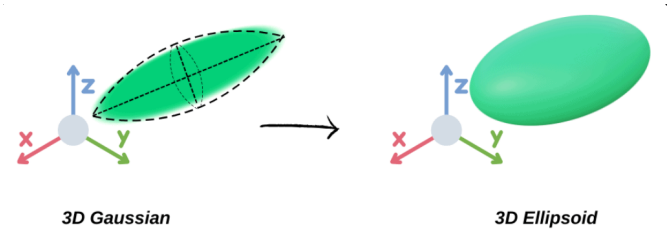


Fig. 6. A 3D gaussian is a soft blob. Its surface at one standard deviation is an ellipsoid.

The shape of a gaussian is stored in its covariance matrix $\boldsymbol{\Sigma}$. It is built from the scale matrix $\mathbf{S} = \operatorname{diag}(\mathbf{s})$ and the rotation matrix $\mathbf{R}(\mathbf{q})$:

$$\boldsymbol{\Sigma} = \mathbf{R}\,\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}. \tag{1}$$

Fig. 7 gives the two matrices. Fig. 8 draws the result for the first gaussian of Fig. 3. The three sizes s_0, s_1, s_2 are the semi-axes of the ellipsoid. The quaternion turns these axes in space. Section 3.2 projects this ellipsoid to the screen.

Listing 2. `render/shaders/splat_prep.vert`: building $\Sigma = \mathbf{R}\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}$

```
vec3 scale = vec3(unpackHalf(t1.y), halfToFloat(t1.z & 65535u));
vec4 q = normalize((vec4(uvec4(t1.w, t1.w >> 8u, t1.w >> 16u, t1.w >>
↪  24u)
                         & 255u) - 128.0) / 127.0);
float w = q.x, x = q.y, y = q.z, z = q.w;
mat3 R = mat3(1.0 - 2.0 * (y * y + z * z), 2.0 * (x * y + w * z), 2.0
↪  * (x * z - w * y),
              2.0 * (x * y - w * z), 1.0 - 2.0 * (x * x + z * z), 2.0
↪  * (y * z + w * x),
              2.0 * (x * z + w * y), 2.0 * (y * z - w * x), 1.0 - 2.0
↪  * (x * x + y * y));
mat3 M = R * mat3(scale.x, 0.0, 0.0, 0.0, scale.y, 0.0, 0.0, 0.0,
↪ scale.z);
mat3 sigma = M * transpose(M);
```

$$R = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{bmatrix}$$

$$S = \begin{bmatrix} s_0 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_2 \end{bmatrix}$$

Fig. 7. The rotation matrix $\mathbf{R}$ from the quaternion (q_w, q_x, q_y, q_z) and the scale matrix $\mathbf{S}$.

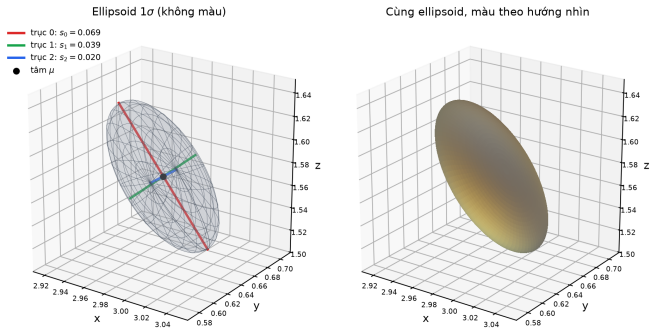


Fig. 8. One gaussian from the file. Left: the 1σ ellipsoid with its three axes and its centre $\boldsymbol{\mu}$. Right: the same ellipsoid with its view-dependent colour.

2. T1: Transformations and hierarchical models

This chapter has two parts. Both move a whole layer.

- **Part 1, layer transformation.** Put a gaussian of a `.ply` file into the world.
- **Part 2, rotation about a point.** Rotate a layer around a chosen centre.

2.1. Part 1: Layer transformation

Each `.ply` file is a layer. A layer has a translation vector $\mathbf{t}$, three Euler angles and a scale factor s. They form the matrix $\mathbf{M}$. This matrix takes a gaussian from file space to world space:

$$\mathbf{p}_w = s\mathbf{A}\mathbf{p}_f + \mathbf{t}, \qquad \boldsymbol{\Sigma}_w = s^2\mathbf{A}\,\boldsymbol{\Sigma}_f\,\mathbf{A}^\top. \tag{2}$$

Here $\mathbf{A} = \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x\mathbf{F}$. $\mathbf{F} = \mathrm{diag}(-1,-1,1)$ is an optional flip for COLMAP files, whose Y axis points down. The gaussian data does not change. Only $\mathbf{M}$ changes. So moving, rotating or scaling a layer is instant. On save, the program writes the transform into every gaussian. It also rotates the SH coefficients so the view-dependent colour stays correct.

Listing 3. `core/math3d.py` and `core/layer.py`: the layer matrix $\mathbf{M}$

```
rx = np.array([[1, 0, 0], [0, cx, -sx], [0, sx, cx]])
ry = np.array([[cy, 0, sy], [0, 1, 0], [-sy, 0, cy]])
rz = np.array([[cz, -sz, 0], [sz, cz, 0], [0, 0, 1]])
return rz @ ry @ rx

rot = self.transform.rotation_matrix() @ (FLIP if flip else np.
↪ identity(3))
m = np.identity(4)
m[:3, :3] = s * rot
m[:3, 3] = t
```

2.2. Part 2: Rotation about a point

A layer can rotate about a point $\mathbf{c}$ with a matrix $\boldsymbol{\Delta}$:

$$\mathbf{R}' = \boldsymbol{\Delta}\mathbf{R}, \qquad \mathbf{t}' = \boldsymbol{\Delta}(\mathbf{t}-\mathbf{c}) + \mathbf{c}. \tag{3}$$

Listing 4. `core/transform.py`, `Transform.rotate_about`

```
def rotate_about(self, delta, pivot):
    pivot = np.asarray(pivot, float)
    self.rotation = matrix_to_euler(delta @ self.rotation_matrix())
    self.position = list(delta @ (self.translation() - pivot) + pivot
↪ )
```

3. T2: Camera and projection

A gaussian goes through four steps to become pixels on the screen [4]. Each step is one box in Fig. 10, read from left to right.

- **Part 1, viewing transformation.** Move the gaussian from object space to camera space.
- **Part 2, projective mapping.** Turn the 3D ellipsoid into a 2D ellipse.
- **Part 3, shading and integration.** Compute the opacity of each pixel and blend the gaussians together.
- **Part 4, viewport transformation.** Place the result in the viewport of the window.
- **Part 5, three cameras.** The camera gives the input of part 1.

The five parts below follow this order.

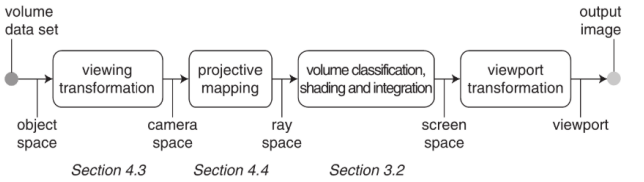


Fig. 10. The splatting pipeline of EWA [4]. The viewing transformation is $\mathbf{W}$ and the projective mapping is approximated by $\mathbf{J}$.

3.1. Part 1: Viewing transformation

This step changes the coordinate system. After it, the camera is at the origin and looks along the $-z$ axis. The `look_at` function builds three camera axes from the eye position $\mathbf{e}$ and the target: right $\mathbf{r}$, up $\mathbf{u}$ and forward $\mathbf{f}$. The view matrix is

$$\mathbf{V} = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{r}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{4}$$

The centre of the gaussian is multiplied by $\mathbf{V}$. The result is the centre in camera space (v_x, v_y, v_z). The shape of the gaussian is only rotated. The matrix $\mathbf{W}$ is the rotation part of $\mathbf{V}$. It is the top-left 3×3 block, in the red box of (4). Its rows are the three camera axes:

$$\mathbf{W} = \begin{pmatrix} r_x & r_y & r_z \\ u_x & u_y & u_z \\ -f_x & -f_y & -f_z \end{pmatrix}. \tag{5}$$

The ellipsoid (1) in camera space is $\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^\top$.

Listing 5. `core/math3d.py`, `look_at`: view matrix $\mathbf{V}$

```
def look_at(eye, target, up):
    forward = target - eye
    forward /= np.linalg.norm(forward)
    right = np.cross(forward, up)
    right /= np.linalg.norm(right)
    true_up = np.cross(right, forward)
    m = np.identity(4, np.float32)
    m[0, :3] = right
    m[1, :3] = true_up
    m[2, :3] = -forward
    m[:3, 3] = -m[:3, :3] @ eye
    return m
```

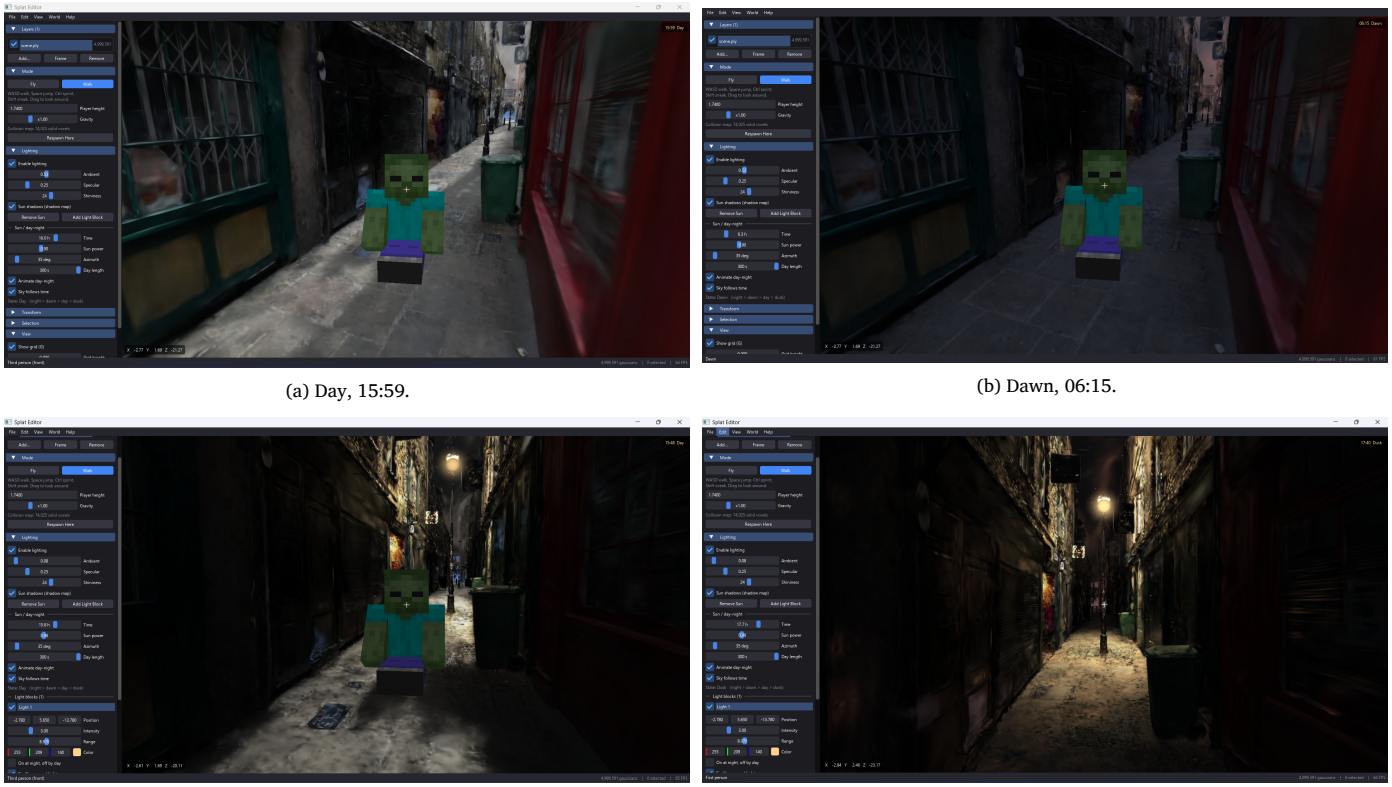
3.2. Part 2: Projective mapping

This step takes camera space to the image plane.

Projecting a point. The perspective matrix uses the vertical field of view φ ($k = 1/\tan(\varphi/2)$), the aspect ratio a, the near plane n and the far plane f:

$$\mathbf{P} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \tag{6}$$

The point is multiplied by $\mathbf{P}$ and divided by c_w. The result is



(a) Day, 15:59.

(b) Dawn, 06:15.

(c) Ambient 0.08 and a lamp.

(d) Dusk, 17:40.

Fig. 9. A scene with the sun, one point light and the character at different times of day.

the normalised coordinates (n_x, n_y) in the range $[-1, 1]$. The division by c_w makes far objects look smaller. The centre of the gaussian is projected this way.

Listing 6. `core/math3d.py`, `perspective`: projection matrix **P**

```
def perspective(fov_y_deg, aspect, near, far):
    zoom = 1.0 / math.tan(math.radians(fov_y_deg) / 2)
    m = np.zeros((4, 4), np.float32)
    m[0, 0] = zoom / aspect
    m[1, 1] = zoom
    m[2, 2] = (far + near) / (near - far)
    m[2, 3] = 2 * far * near / (near - far)
    m[3, 2] = -1
    return m
```

Projecting a gaussian. A gaussian is not a point. Its ellipsoid must become an ellipse on screen. The division by c_w is not linear, so we cannot multiply **P** with the ellipsoid. EWA replaces the projection by its derivative (the Jacobian) at the centre of the gaussian:

$$\mathbf{J} = \begin{pmatrix} \frac{f_{px}}{-v_z} & 0 & \frac{f_{px}\, v_x}{v_z^2} \\ 0 & \frac{f_{px}}{-v_z} & \frac{f_{px}\, v_y}{v_z^2} \\ 0 & 0 & 0 \end{pmatrix}. \tag{7}$$

Here $f_{px} = kH/2$ is the focal length in pixels, with k from (6) and H the viewport height. Joining step 1 and step 2, the 2D covariance is

$$\mathbf{\Sigma}' = \mathbf{J}\,\mathbf{W}\,\mathbf{\Sigma}\,\mathbf{W}^{\top}\mathbf{J}^{\top}. \tag{8}$$

Here $\mathbf{\Sigma}$ is the 3D covariance (1). $\mathbf{W}$ is the rotation part of the view matrix from step 1. $\mathbf{J}$ is the Jacobian (7). Let the top-left 2×2 block of $\mathbf{\Sigma}'$ be $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$. Its eigenvalues are

$$\lambda_{1,2} = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}. \tag{9}$$

The two semi-axes of the ellipse are proportional to $\sqrt{\lambda_1}$ and $\sqrt{\lambda_2}$. The shader draws each gaussian as a quad covering at most 3 standard deviations. It drops gaussians whose centre is behind the camera or more than 1.3 times outside the screen.

Listing 7. `render/shaders/splat_prep.vert`: Jacobian $\mathbf{J}$ and $\mathbf{\Sigma}' = \mathbf{J}\mathbf{W}\mathbf{\Sigma}\mathbf{W}^{\top}\mathbf{J}^{\top}$

```
float vx = viewPos.x, vy = viewPos.y, vz = viewPos.z;
mat3 J = mat3(uFocal / -vz, 0.0, 0.0,
              0.0, uFocal / -vz, 0.0,
              uFocal * vx / (vz * vz), uFocal * vy / (vz * vz), 0.0);
mat3 T = J * mat3(uModelView) * linear;
mat3 cov = T * sigma * transpose(T);
float a = cov[0][0] + 0.3;
float b = cov[0][1];
float c = cov[1][1] + 0.3;
```

Listing 8. `render/shaders/splat_prep.vert`: eigenvalues and the two ellipse axes

```
float mid = 0.5 * (a + c);
float radius = length(vec2(0.5 * (a - c), b));
float lambda1 = mid + radius;
float lambda2 = max(mid - radius, 0.1);
vec2 axis1 = (abs(b) < 1e-6) ? (a >= c ? vec2(1.0, 0.0) : vec2(0.0,
↪ 1.0))
                             : normalize(vec2(b, lambda1 - a));
vec2 axis2 = vec2(-axis1.y, axis1.x);
vec2 major = min(extent * sqrt(lambda1), 1024.0) * axis1 * toNdc;
vec2 minor = min(extent * sqrt(lambda2), 1024.0) * axis2 * toNdc;
```

3.3. Part 3: Shading and alpha blending

This step gives each pixel a colour. The fragment shader visits each pixel $\mathbf{x}$ of the quad. It evaluates the 2D Gaussian function at $\mathbf{x}$. The centre $\boldsymbol{\mu}'$ is the projected gaussian centre. The covariance $\mathbf{\Sigma}'$ comes from (8):

$$G(\mathbf{x}) = \exp\!\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu}')^{\top}\,\mathbf{\Sigma}'^{-1}\,(\mathbf{x}-\boldsymbol{\mu}')\Big). \tag{10}$$

The exponent is $-r^2/2$. Here r is the distance to the ellipse centre, in standard deviations. The opacity of gaussian i at the

pixel is $a_i = \alpha_i\, G_i(\mathbf{x})$.

Listing 9. render/shaders/splat.frag: $a = \alpha\, e^{-r^2/2}$

```
float r2 = dot(vCorner, vCorner);
if (r2 > 9.0) discard;
float alpha = vColor.a * exp(-0.5 * r2);
if (alpha < 1.0 / 255.0) discard;
fragColor = vec4(vColor.rgb * alpha, alpha);
```

The gaussians are drawn from far to near and blended. The pixel colour is

$$\mathbf{C} = \sum_i \mathbf{c}_i\, a_i\, T_i, \qquad T_i = \prod_{j<i} (1 - a_j), \tag{11}$$

Here the gaussians are numbered from near to far. T_i is the transmittance. It is the fraction of light left after passing through the gaussians nearer than i.

The program draws from far to near. It gets the same result with the blend $\mathbf{C} \leftarrow \mathbf{c}\, a + (1 - a)\, \mathbf{C}$ for each splat. Here $\mathbf{C}$ is the accumulated colour.

The result depends on the order. So we must sort by depth. The program converts depth to 16 bits. It sorts with NumPy in a background thread. Until the sort is done, it draws with the order of the previous sort. The depth test is turned off. The reason is that a transparent gaussian must not hide the gaussians behind it.

Listing 10. render/splat_renderer.py: the blend $\mathbf{C} \leftarrow \mathbf{c}a + (1-a)\mathbf{C}$

```
GL.glDisable(GL.GL_DEPTH_TEST)
GL.glEnable(GL.GL_BLEND)
GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
GL.glDrawArraysInstanced(GL.GL_TRIANGLE_STRIP, 0, 4, self.draw_count)
```

Listing 11. render/sorter.py, DepthSorter.sort: far-to-near order

```
depth[start:start + count] = positions[start:start + count] @ row[:3]
    + row[3]
visible = np.flatnonzero(depth < 0)
far_z, near_z = float(depth.min()), float(depth.max())
scale = 65535.0 / (near_z - far_z) if near_z > far_z else 0.0
depth_key = ((depth - far_z) * scale).astype(np.uint16)
return visible[np.argsort(depth_key, kind='stable')].astype(np.uint32
    )
```

3.4. Part 4: Viewport transformation

The last step converts the normalised coordinates (n_x, n_y) of step 2 to pixels. The viewport has size $W \times H$ and corner (x_0, y_0):

$$x_{px} = x_0 + \frac{n_x + 1}{2} W, \qquad y_{px} = y_0 + \frac{1 - n_y}{2} H. \tag{12}$$

OpenGL does this step itself when drawing. The program also uses this formula to know which pixel a gaussian is on, for example in brush selection (Section 7).

Listing 12. editor/brush.py, BrushTool._to_screen: clip space to pixels

```
def _to_screen(clip, viewport):
    left, top, width, height = viewport
    x = left + (clip[..., 0] / clip[..., 3] + 1) * 0.5 * width
    y = top + (1 - clip[..., 1] / clip[..., 3]) * 0.5 * height
    return x, y
```

3.5. Part 5: Three cameras

Orbit camera. The camera always looks at a point $\mathbf{t}$. Its position uses a yaw angle θ, a pitch angle ϕ and a distance d:

$$\mathbf{e} = \mathbf{t} + d\,(\cos\phi \sin\theta,\ \sin\phi,\ \cos\phi\cos\theta). \tag{13}$$

The pitch is limited to about $\pm 89°$ so the camera does not flip.

Listing 13. render/camera.py, OrbitCamera.position

```
PITCH_LIMIT = 1.55
cp = math.cos(self.pitch)
offset = np.array([cp * math.sin(self.yaw), math.sin(self.pitch),
                   cp * math.cos(self.yaw)])
return self.target + self.distance * offset
```

Fly camera. The velocity does not change at once. It moves gradually to the wanted velocity $\mathbf{v}^*$, so the camera has inertia:

$$\mathbf{v} \leftarrow \mathbf{v} + (\mathbf{v}^* - \mathbf{v})\min(10\Delta t, 1), \qquad \mathbf{e} \leftarrow \mathbf{e} + \mathbf{v}\,\Delta t. \tag{14}$$

Listing 14. render/camera.py, OrbitCamera.fly

```
wanted = direction * self.fly_speed * (self.SPRINT_FACTOR if sprint
    else 1.0)
self.velocity += (wanted - self.velocity) * min(self.FLY_ACCEL * dt,
    1.0)
self.target += self.velocity * dt
```

4. T3: Texture mapping

The program uses a texture to paint the box model of the character with a skin image. The character has 6 boxes (head, body, two arms and two legs). The skin is a 64×64 image (Fig. 13) that holds the unfolded faces of all 6 boxes in the Minecraft cross layout. Each face of a box only has to take its own cell of the image. There are three steps (Fig. 11).

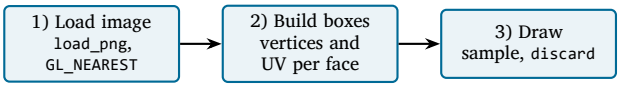


Fig. 11. Three steps to draw the skin.

1) Build the box. A box has centre $\mathbf{c}$, size (w, h, d) and inflation δ. Each corner of a face is $\mathbf{p} = \mathbf{c} + \big(s_x(\frac{w}{2} + \delta),\ s_y(\frac{h}{2} + \delta),\ s_z(\frac{d}{2} + \delta)\big)$, $s_x, s_y, s_z \in \{\pm 1\}$. The box is built twice: a base layer ($\delta = 0$) and an overlay layer ($\delta = 0.25$ for body and limbs, 0.5 for the hat). The overlay is slightly larger, so it does not stick to the base layer.

Listing 15. render/avatar_renderer.py, _box: corner $\mathbf{p}$ of a face

```
(cx, cy, cz), (w, h, d) = centre, size
hx, hy, hz = w / 2 + inflate, h / 2 + inflate, d / 2 + inflate
...
for (sx, sy, sz), (u, v) in zip(_FACES[face], uvs):
    vertices.append((cx + sx * hx, cy + sy * hy, cz + sz * hz, ...))
```

2) Pick the cell in the image. A box has origin (o_x, o_y) in the skin. Fig. 12 shows where the six faces are. For example, the head has origin $(0, 0)$ and $w = h = d = 8$, so its front face (the face of the character) is in the cell $(8, 8, 8, 8)$. The four corners of a cell (r_x, r_y, r_w, r_h) get the UV

$$(u, v) = \Big(\frac{r_x + k\, r_w}{64},\ \frac{r_y + l\, r_h}{64}\Big), \quad (k, l) \in \{(0,0), (1,0), (1,1), (0,1)\}. \tag{15}$$

The order of the four corners must match the order of the four vertices of the face. If they differ, the image is flipped or rotated.

Listing 16. render/avatar_renderer.py: skin cell and UV of a face

```
def _uv_region(face, ox, oy, w, h, d):
    return ((ox + d, oy + d, w, h), (ox + 2 * d + w, oy + d, w, h),
            (ox, oy + d, d, h), (ox + d + w, oy + d, d, h),
            (ox + d, oy, w, d), (ox + d + w, oy, w, d))[face]

rx, ry, rw, rh = _uv_region(face, ox, oy, w, h, d)
uvs = ((rx, ry), (rx + rw, ry), (rx + rw, ry + rh), (rx, ry + rh))
...  u / SKIN_SIZE, v / SKIN_SIZE
```

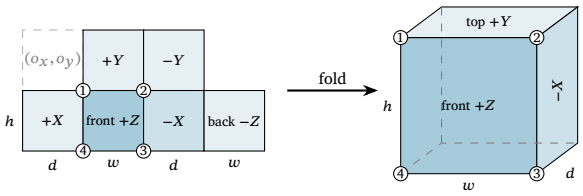


Fig. 12. A 3D box and its cross layout. The $+Z$ face ($w \times h$) has the cell $(o_x{+}d,\ o_y{+}d,\ w,\ h)$ and the $+Y$ face has the cell $(o_x{+}d,\ o_y,\ w,\ d)$. The corners 1, 2, 3, 4 of each face get $(k, l) = (0,0), (1,0), (1,1), (0,1)$ in (15).

3) Draw. The skin is loaded by my own `load_png` function and filtered with `GL_NEAREST`, so the pixels stay sharp ($\lfloor 64u \rfloor, \lfloor 64v \rfloor$). The fragment shader discards pixels with $\alpha < 0.5$, so the transparent parts of the overlay show the base layer. The final colour is the texel times $0.62 + 0.38 \max(\mathbf{n}\cdot\mathbf{l}, 0)$. Each box is drawn with its own model matrix, so arms and legs rotate independently about their joints.



Fig. 13. The 64×64 skin of the character. Each box (head, body, arms, legs) has one cross for the base layer and one for the overlay (hat, jacket, sleeves).

5. T4: Lighting and shadows

The final colour of a gaussian is made in four parts.

- **Part 1, view-dependent colour.** Read the base colour from the SH coefficients.
- **Part 2, Phong lighting.** Add ambient, diffuse and specular light.
- **Part 3, the sun.** Move the sun and the sky colour with the hour.
- **Part 4, shadows.** Test each gaussian against a shadow map.

5.1. Part 1: View-dependent colour

The colour of a gaussian changes with the view direction $\mathbf{d}$. 3DGS stores this with spherical harmonics (SH) coefficients [2]:

$$\mathbf{c}(\mathbf{d}) = \mathrm{clip}\Big(\mathbf{c}_0 + \sum_{k=1}^{K} Y_k(\mathbf{d})\,\mathbf{h}_k,\ 0,\ 1\Big). \tag{16}$$

Here $\mathbf{c}_0$ is the base colour, $\mathbf{h}_k$ are the coefficients in the file and Y_k are the basis functions. The coefficients are in file space. So the shader rotates the view direction into that space: $\mathbf{d} = \mathrm{norm}(\mathbf{A}^{\top}(\mathbf{p}_w - \mathbf{e}))$. It does not have to rotate the coefficients every frame. Fig. 14 shows the idea for degrees 0 to 2. Each basis function is a pattern on the sphere. The weighted sum gives one colour for each view direction.

Listing 17. `render/shaders/splat_prep.vert`: view-dependent SH colour

```
vec3 c = -0.4886025119029199 * y * shCoeff(splat, 0)
       +  0.4886025119029199 * z * shCoeff(splat, 1)
       -  0.4886025119029199 * x * shCoeff(splat, 2);
...
vec3 color = vec3(rg, ba.x);
if (uShDegree > 0)
    color += shColor(splat, normalize(transpose(linear)
                                      * (center - uCamPos)));
color = clamp(color, 0.0, 1.0);
```

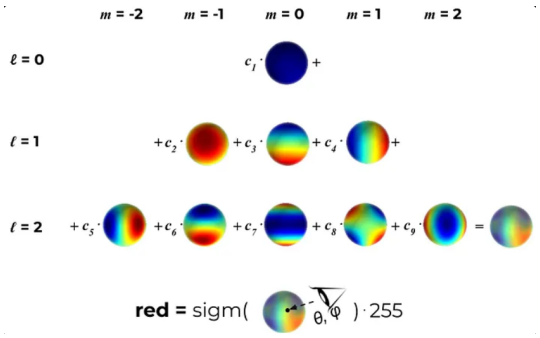


Fig. 14. Spherical harmonics up to degree 2. The nine basis functions are weighted by the coefficients $c_1..c_9$ and added. The colour is read at the view direction (θ, φ).

5.2. Part 2: Phong lighting

Lighting is computed once per gaussian. The normal $\mathbf{n}$ is the shortest axis of the ellipsoid, flipped towards the camera. The final colour uses the Phong model [5]. It has three parts (Fig. 15). Let $\mathbf{c}$ be the colour of the gaussian.

Ambient. The ambient light $\mathbf{L}_a$ reaches every gaussian equally:

$$\mathbf{I}_a = \mathbf{c}\,\mathbf{L}_a. \tag{17}$$

Diffuse. A surface that faces the light is brighter. For light j, $\mathbf{L}_j$ is its colour and $\mathbf{l}_j$ is the direction to the light:

$$\mathbf{I}_d = \mathbf{c} \sum_j \mathbf{L}_j\, w(\mathbf{n}, \mathbf{l}_j)\, S_j, \qquad w(\mathbf{n}, \mathbf{l}) = \mathrm{clip}\Big(\frac{\mathbf{n}\cdot\mathbf{l} + 0.35}{1.35}, 0, 1\Big). \tag{18}$$

The term w is a softened $\mathbf{n}\cdot\mathbf{l}$, because the estimated normal is noisy.

Specular. A highlight appears where the reflected ray $\mathbf{r}_j$ is close to the direction $\mathbf{v}$ to the camera:

$$\mathbf{I}_s = k_s \sum_j \mathbf{L}_j\, (\mathbf{r}_j \cdot \mathbf{v})_+^m\, S_j. \tag{19}$$

Here k_s is the specular strength and m is the shininess. This part does not use the colour $\mathbf{c}$. It is zero when $\mathbf{n}\cdot\mathbf{l}_j \le 0$.

Sum. The final colour is the sum of the three parts:

$$\mathbf{c}_{out} = \mathbf{I}_a + \mathbf{I}_d + \mathbf{I}_s. \tag{20}$$

S_j is the shadow factor of the sun, or the distance falloff of a point light. The scene has one sun and up to 8 point lights. Point lights cast no shadows.

Listing 18. `render/shaders/splat_prep.vert`, `shade`: Phong lighting

```
vec3 diffuse = uAmbient;
vec3 spec = vec3(0.0);
float ndl = dot(n, uSunDir);
float shadow = shadowFactor(worldPos, n);
diffuse += uSunColor * shadow
         * clamp((ndl + 0.35) / 1.35, 0.0, 1.0);
if (ndl > 0.0)
    spec += uSunColor * shadow * pow(max(dot(
        reflect(-uSunDir, n), v), 0.0), uShininess);
...
return albedo * diffuse + spec * uSpecular;
```

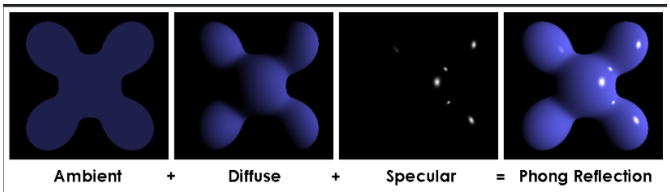


Fig. 15. The Phong model [5]: ambient, diffuse and specular parts, and their sum.

5.3. Part 3: The sun

The sun direction is computed from the hour h of the day. The sun rises at 6, is highest at 12 and sets at 18. The sky colour and the ambient light change gradually between day and night. They are slightly orange at dusk (Fig. 9).

5.4. Part 4: Shadows

The program draws the centres of the solid gaussians as points into a 2048^2 depth map seen from the sun [6]. Each gaussian is projected into this map to compare depths. If something is closer to the sun, the gaussian is in shadow. The program averages 3×3 samples to soften the shadow edge (PCF [7]):

$$S = \frac{1}{9} \sum_{i,j=-1}^{1} \left[z_s - b \le D_s(u_s + i\tau,\, v_s + j\tau) \right]. \tag{21}$$

Here (u_s, v_s, z_s) are the coordinates of the gaussian in the map, D_s is the stored depth, τ is the texel step and b is the bias. The bias stops a gaussian from shadowing itself.

Listing 19. `render/shaders/splat_prep.vert`, `shadowFactor`: 3×3 PCF

```
vec4 lp = uLightVP * vec4(worldPos + n * uShadowTexel * 2.0, 1.0);
vec3 s = lp.xyz / lp.w * 0.5 + 0.5;
float lit = 0.0;
for (int i = -1; i <= 1; i++)
    for (int j = -1; j <= 1; j++) {
        vec2 uv = s.xy + vec2(i, j) * uShadowInv;
        float d = textureLod(uShadowMap, uv, 0.0).r;
        lit += (s.z - uShadowBias <= d) ? 1.0 : 0.0;
    }
return lit / 9.0;
```

Listing 20. `core/lighting.py`, `Lighting.shadow_setup`: view matrix from the sun

```
d = self.sun_direction()
r = self.radius * 1.1
eye = self.center + d * 2.0 * r
view = look_at(eye, self.center, up)
proj = orthographic(r, 0.01 * r, 4.0 * r)
return (proj @ view).astype(np.float32), texel, bias
```

6. T5: Animation and simulation

Every motion is multiplied by the frame time Δt. So the speed depends little on the machine. There are three parts, in the order of one frame.

- **Part 1, day–night cycle.** Advance the clock and move the sun.
- **Part 2, voxel collision.** Turn the gaussians into solid voxels.
- **Part 3, the player.** Walk, jump and fall on the voxels.

6.1. Part 1: Day–night cycle

This is a state machine with 4 states: NIGHT → DAWN → DAY → DUSK. The clock runs by itself. Lights set to "on at night" turn on at dusk and off at dawn. The states start at hour 5 (DAWN), 7 (DAY), 17 (DUSK) and 19 (NIGHT). One day lasts $T_{day} = 36$ s, so the hour advances by $\Delta h = 24\,\Delta t / T_{day}$ each frame. The sun position follows the hour:

$$\theta = \frac{h - 6}{12}\pi, \qquad \epsilon = 65^\circ \sin\theta, \quad \mathbf{s} = \big(\cos\epsilon\cos\phi,\ \sin\epsilon,\ \cos\epsilon\sin\phi\big), \tag{22}$$

where $\phi = \phi_0 + \theta - \pi/2$ and ϕ_0 is the sun azimuth. The sun rises at 6, is highest at 12 and sets at 18. The daylight factor is $D = \text{smoothstep}(-0.08,\, 0.3,\, s_y)$. The ambient colour and the sky colour are mixed between a night value and a day value with D, and the sky gets an orange tint near the horizon.

Listing 21. `core/lighting.py`: hour, sun direction and daylight factor

```
hours_per_second = 24.0 / max(self.day_length, 1.0)
self.hour = (self.hour + dt * hours_per_second) % 24.0

angle = (self.hour - 6.0) / 12.0 * math.pi
elevation = math.sin(angle) * math.radians(MAX_SUN_ELEVATION)
azimuth = (math.radians(self.sun_azimuth)
           + (angle - math.pi / 2))
return np.array([math.cos(elevation) * math.cos(azimuth),
                 math.sin(elevation),
                 math.cos(elevation) * math.sin(azimuth)])

return _smooth(-0.08, 0.3, self.sun_direction()[1])
```

6.2. Part 2: Collision with voxels

A gaussian has no clear surface, so direct collision is hard. The program divides space into voxels of edge v. A voxel with enough gaussians is solid. Each solid voxel is stored as one 64-bit integer:

$$\mathbf{c} = \left\lfloor \frac{\mathbf{p} - \mathbf{o}}{v} \right\rfloor, \qquad \text{key} = (c_x \ll 42) \,|\, (c_z \ll 21) \,|\, c_y. \tag{23}$$

The keys are sorted. Because c_y is in the low bits, the cells of one column sit next to each other. Finding the ground under the feet takes one `np.searchsorted` call.

Listing 22. `core/physics.py`, `CollisionWorld`: voxel keys and ground search

```
cells = np.floor((positions - self.origin) / self.voxel).astype(np.
↪ int64)
keys = (cells[:, 0] << (2 * _BITS)) | (cells[:, 2] << _BITS) | cells
↪ [:, 1]
self.keys = np.unique(keys)

idx = np.searchsorted(self.keys, query, side='right') - 1
```

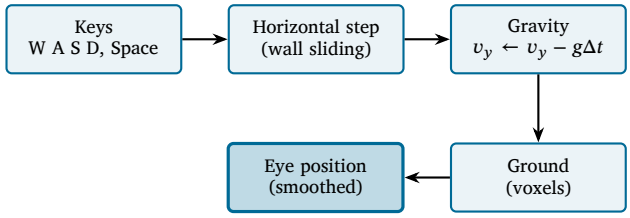


Fig. 16. One frame of walk mode.

6.3. Part 3: The player

Fig. 16 summarises one frame. The player is a cylinder. At a wall, the player slides along it. The player can climb a low step and can jump.

Gravity. When not on the ground, the program integrates with Euler steps:

$$v_y \leftarrow v_y - g\,\Delta t, \qquad y \leftarrow y + v_y\,\Delta t. \tag{24}$$

Scale and jump. All sizes and speeds scale with the player height H. The reason is that a 3DGS scene has no fixed unit. The constants are: voxel edge $v = H/8$, player radius $0.14H$, step height $0.35H$, gravity $g = 12H$, walking speed $2.5H$ per second (sprint ×1.7, sneak ×0.4). A jump reaches the height

$0.75H$, so the take-off speed is

$$v_0 = \sqrt{2\,g\,(0.75H)}. \tag{25}$$

Eye smoothing. The eye height is smoothed while the player stands on the ground, so the rough gaussian surface does not shake the camera.

Listing 23. `core/physics.py`, `Physics.walk`: jump and gravity

```
if player.on_ground and jump:
    player.vel_y = math.sqrt(2.0 * self.gravity * self.jump_ratio * h
↪ )
    player.on_ground = False
if not player.on_ground:
    player.vel_y -= self.gravity * dt
    new_y = y + player.vel_y * dt
```

7. T6: Interaction and GUI

The interaction has four parts, from the input to the screen.

- **Part 1, event handling.** Decide who receives each mouse or keyboard event.
- **Part 2, brush selection.** Select gaussians by painting on the screen.
- **Part 3, soft delete and undo.** Delete the selection and restore it.
- **Part 4, interface.** Panel, HUD and shortcuts.

7.1. Part 1: Event handling

Mouse and keyboard events go to ImGui first. If ImGui does not use them, they go to the camera, the selection brush and the shortcuts.

7.2. Part 2: Brush selection

Press `B` to turn on the brush. Paint on the screen to select gaussians. They turn yellow. The centre of each gaussian is projected to a pixel $\mathbf{p}$ (Section 3.1).

Capsule test. A stroke from $\mathbf{a}$ to $\mathbf{b}$ is a capsule of radius r. A gaussian is selected if

$$\|\mathbf{p}-\mathbf{a}-\tau(\mathbf{b}-\mathbf{a})\| \le r, \quad \tau = \operatorname{clip}\frac{(\mathbf{p}-\mathbf{a})\cdot(\mathbf{b}-\mathbf{a})}{\|\mathbf{b}-\mathbf{a}\|^2} \in [0,1]. \tag{26}$$

That is, the distance from $\mathbf{p}$ to the segment $\mathbf{ab}$ is at most r.

Listing 24. `editor/brush.py`, `BrushTool._segment_dist2`: capsule test

```
def _segment_dist2(p, a, b):
    ab = b - a
    length2 = float(ab @ ab)
    t = (np.clip((p - a) @ ab / length2, 0, 1) if length2 > 0
         else np.zeros(p.shape[:-1]))
    nearest = a + t[..., None] * ab
    diff = p - nearest
    return (diff * diff).sum(axis=-1)
```

Properties. The brush tests only the gaussian centre and does not test depth. So it selects through the scene, like SuperSplat [3]. For speed, the brush rejects a whole block of 1024 gaussians when the bounding box of the block is outside the stroke.

7.3. Part 3: Soft delete and undo

Each gaussian has one state byte on the GPU with two bits: *selected* and *deleted*. The `Delete` command only sets the "deleted" bit. The shader skips these gaussians. The original data does not change. Each delete is one command in `History` (Command pattern [8], at most 20 steps). `Ctrl+Z` restores the last command.

Listing 25. `core/layer.py` and `core/history.py`: soft delete and undo

```
def delete_selected(self):
    indices = np.flatnonzero(self.selection)
    self.deleted[indices] = True
    self.selection[indices] = False
    return indices

def push(self, command):
    self._stack.append(command)
    del self._stack[:-self.max_steps]

def undo(self):
    command = self._stack.pop()
    command.undo()
```

7.4. Part 4: Interface

The ImGui panel on the left controls layers, transforms, selection, camera mode and lighting. The HUD shows the crosshair, the clock, the position and the brush circle. Table 3 lists the main keys.

Table 3. Main controls.

Key / mouse	Function
Left / middle drag	Orbit / pan
Right drag + `WASD`	Look around, fly (`Ctrl` = fast)
`M`, `F5`, `C`, `V`	Fly/walk; third person; sit; wave
`F`, `G`	Frame the scene; show/hide grid
`B`, `[`, `]`	Brush; brush size
`Delete`, `Ctrl+Z`	Delete / undo
`Ctrl+O/I/S`	Open / add layer / save

8. Limitations and future work

8.1. Limitations

- **Speed depends on the CPU.** Depth is sorted with NumPy in a background thread (Section 3.3). When the camera turns fast the image can flicker, and large scenes must limit the number of drawn gaussians (LOD).
- **Approximate geometry.** A gaussian has no real surface. The brush, collision, normals and shadows all use the gaussian centres, so the brush also selects hidden objects, collision can have holes and the lighting is rough.

8.2. Future work

- **Support for simulation.** Use the voxel map and the existing physics to simulate motion inside a scene captured from the real world, for example to test the path of a person, a robot or a vehicle. This needs more accurate collision and moving objects.
- **Use in games.** Turn a 3DGS scene into a game level: add non-player characters, items and quests, and keep the day–night cycle and the lights. The cleaned scene can be exported to a game engine such as Unity or Unreal.

9. AI usage statement

The course allows students to use AI tools (ChatGPT, Gemini) to learn concepts, debug and improve their code. In this project I used ChatGPT and Gemini for two things:

- **Learning concepts.** Explanations of 3D Gaussian Splatting, EWA, spherical harmonics, shadow maps and related graphics techniques.
- **Debugging and improving code.** Finding bugs and suggesting shorter or faster ways to write the program.

I read, checked and tested the content suggested by AI, and I am responsible for all the code and for this report.

10. Conclusion

Splat Editor can read, display, edit and save 3DGS files. The program uses all six technique groups of the course. Table 4 shows where each part is in the source code.

Table 4. Main source files (in `splat_editor/`).

Part	File
PLY read/write	`io/ply.py`, `io/codec.py`
Gaussian rendering	`render/splat_renderer.py`, `render/sorter.py`, `render/shaders/splat_prep.vert`
T1	`core/layer.py`, `core/transform.py`, `core/avatar_pose.py`
T2	`core/math3d.py`, `render/camera.py`
T3	`render/avatar_renderer.py`
T4	`core/sh.py`, `core/lighting.py`, `render/shaders/shadow.vert`
T5	`core/physics.py`, `core/avatar_pose.py`
T6	`editor/brush.py`, `core/history.py`, `ui/`

References

[1] J. L. Schönberger, J.-M. Frahm, Structure-from-motion revisited, in: CVPR, 2016, pp. 4104–4113.
[2] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, 3D Gaussian splatting for real-time radiance field rendering, ACM Trans. Graph. 42(4) (2023) 139.
[3] PlayCanvas, SuperSplat: an open-source 3D Gaussian Splat editor, https://github.com/playcanvas/supersplat.
[4] M. Zwicker, H. Pfister, J. van Baar, M. Gross, EWA splatting, IEEE Trans. Vis. Comput. Graph. 8(3) (2002) 223–238.
[5] B. T. Phong, Illumination for computer generated pictures, Commun. ACM 18(6) (1975) 311–317.
[6] L. Williams, Casting curved shadows on curved surfaces, in: SIGGRAPH, 1978, pp. 270–274.
[7] W. T. Reeves, D. H. Salesin, R. L. Cook, Rendering antialiased shadows with depth maps, in: SIGGRAPH, 1987, pp. 283–291.
[8] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.